

Java Programming 3

Week 1

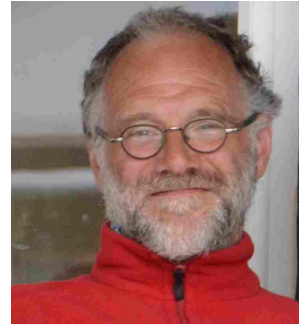
Welcome!

Who are we?

- Hans Vochten
 - Java Programming 3
 - Integration 3

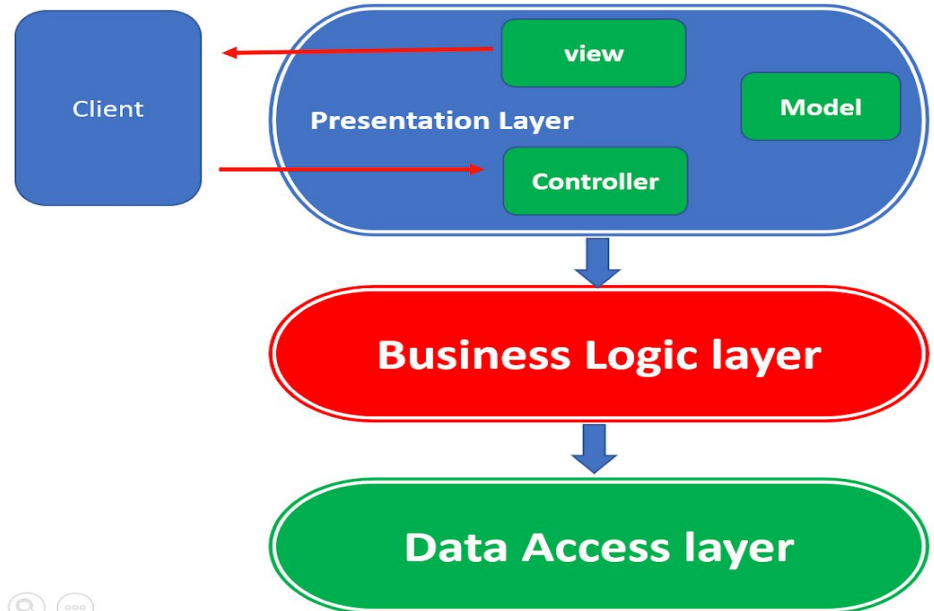
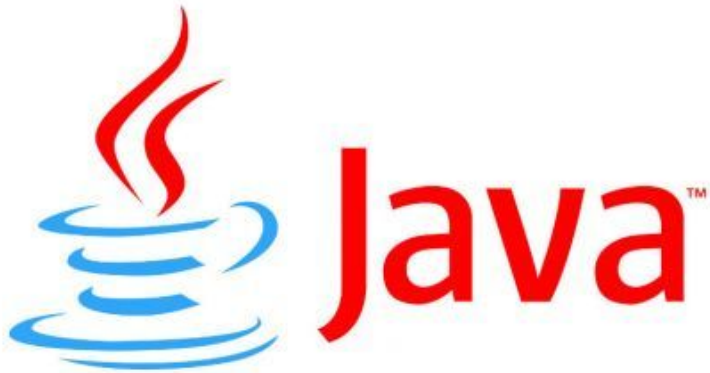


- Jan de Rijke
 - Java Programming 3
 - internship



Programming 3?

- Using Java to build enterprise applications
 - Using the **Spring Framework**
 - Using **Advanced Java Features and Libraries**
 - Using **3-tier architecture**

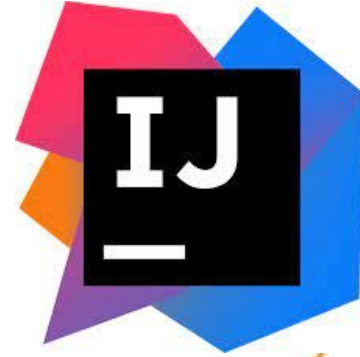


Software?

- IDE: IntelliJ 2023.2 or later
- Java: Oracle JDK 21
- Git

LTS and stable...!

→ Instructions and licenses: see Canvas



git

How?

- 12 weeks - 5 hours/week
- Every week
 - Learn new Spring Framework features
 - Learn new Java Language features or Java libraries
 - Study demos and examples
 - Apply in exercises
 - Apply in your *individual project*
- Every 2 weeks
 - Sprint deliverable of the next part of the *individual project*



Individual project

- Based on your own dataset
- Web application with Java back-end connected to a database
- Possibility to **C**reate, **R**ead, **U**ppdate, **D**eleete items
- Implemented using a 3-tier architecture
- Every week you get a new assignment:
 - Apply the topics of that weeks to the project
- Every 2 weeks you tag a release in gitlab
- The application “*grows*” towards a final product



Individual project: dataset

- You choose your dataset: see Canvas for specs
- Submit your dataset on MS Teams
 - We will review and approve
 - Everyone has different dataset!



Evaluation?

- 6 release tags → for feedback
- Final deliverable @ end of second period
- Oral exam:
 - Demonstration and explanation of your personal CRUD application
 - Oral interrogation about the topics of the course

→ Important: **you need a project to be able to participate in the oral exam!**



Agenda (more or less...)

Spring Framework:

- Spring basics: Dependency Injection
 - the Application Context
- Spring Boot
- Spring MVC with Thymeleaf
- Validation
- JDBC Templates
- Object Relational Mapping with JPA
- ...

Other topics:

- Gradle/Git
- What is a 3-tier architecture?
- Streams
- Annotations
- Generics
- Logging
- ...



Agenda this week

Introduction

Where are we heading to...?

Java project with Gradle

Git

Lambdas and streams



Agenda this week

Introduction

Where are we heading to...?

Java project with Gradle

Git

Lambdas and streams

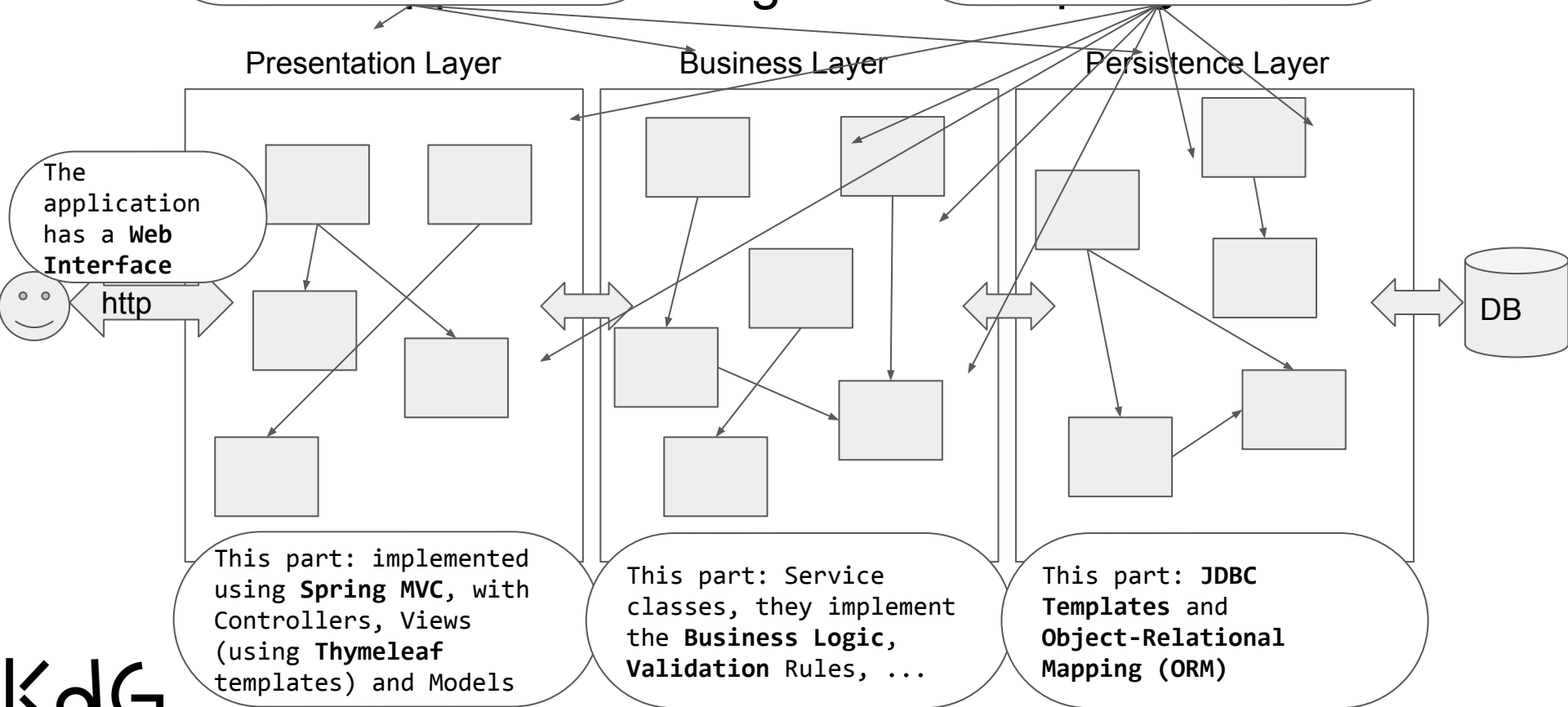
A 3-layer

We use a **3-layered architecture** and think about the dependencies between the layers...

using the Java

A lot of *Components!* (aka *Beans* which are Java Classes). They are managed by a **Spring Container**

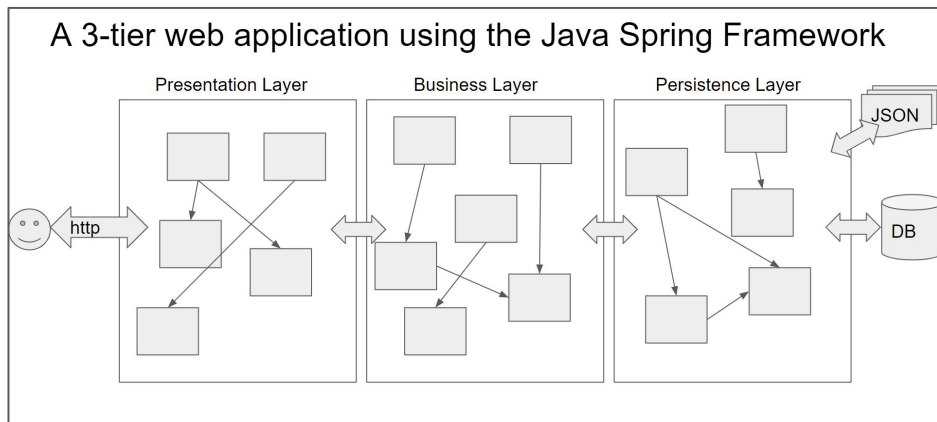
Framework



Before we can start with →

We need some extra Java tooling:

- **Gradle**: build automation
- **Git**: version control
- **Lambdas and Streams**: introduce functional programming and easy collections handling
- **Annotations**: metadata to configure stuff → next week





Agenda this week

Introduction

Where are we heading to...?

Java project with Gradle

Git

Lambdas and streams

Gradle: build automation tool

- Configure and automate the build process
- Gradle defines *tasks* to
 - Download the dependent libraries (“dependencies”)
 - Compile the source code to class files
 - Make jar files
 - Run tests
 - Deploy to the server
 - ...
- Very good integration with IntelliJ!
- Read these tutorials:
 - [What is Gradle?](#)
 - [Building Java Applications Sample](#) (just read this one and try to understand...)



Gradle: build.gradle.kts file (1/2)

- Configuration of the build process for the application
 - Specify the main class
 - Specify the dependencies
 - ...
- Uses the Kotlin syntax...
- Example:

```
plugins {  
    id("java")  
    id("application")  
}  
  
application {  
    mainClass = "be.kdg.programming3.StartApplication"  
}  
  
group = "be.kdg.java3"  
version = "1.0-SNAPSHOT"
```

Gradle uses plugins, you specify them here. We add the "application" plugin to be able to run as a Java application

Here we configure the "application" plugin: we tell it where to find the main class...



Gradle: build.gradle.kts file (2/2)

- Configuration of the build process for the application
 - Specify the main class
 - **Specify the dependencies**
 - ...
- Uses the Kotlin syntax...
- Example:

```
...  
repositories {  
    mavenCentral()  
}
```

```
dependencies {  
    testImplementation(platform("org.junit:junit-bom:5.10.0"))  
    testImplementation("org.junit.jupiter:junit-jupiter")  
}  
...
```

Here you can specify what repository to use for the dependencies.

IntelliJ already added 2 dependencies: the junit libs for testing. We can add more dependencies here...



demo_gradle

- In this demo we will
 - Create a Java - Gradle project in IntelliJ
 - Add a StartApplication class with a main method → “Hello World with Gradle!”
 - Modify the gradle.build file to be able to run the application as a Gradle task
 - Run from Gradle view (IntelliJ) - run from Terminal





Exercise 1

- Create a new project “persons” project (group: be.kdg.programming3)
- Modify the build.gradle to be able to run as a Java Application
- Add a StartApplication class with a main method
- Add a Person class, a person has a tellJoke method
- Add a dependency to the random-joke-crawler (do not take the 0.1 version)
(look up @ <https://search.maven.org/> and look up how to use it @ github.com)
- Implement the tellJoke method in a Person class, run from main method
 - In tellJoke get a joke from the random-joke-crawler
- Run the application from a terminal as a Gradle task



Agenda this week

Introduction

Where are we heading to...?

Java project with Gradle

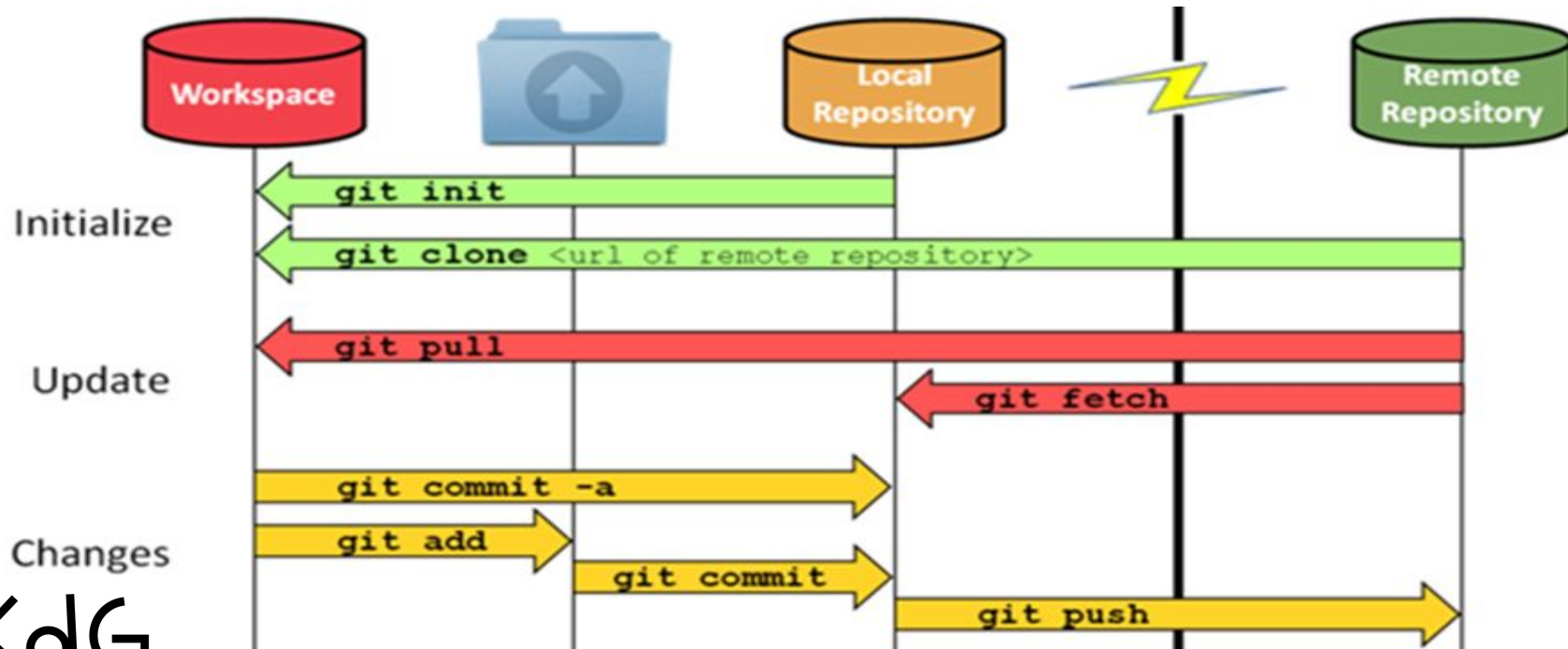
Git

Lambdas and streams



Git: version control

- Basic commands:



demo_git

- In this demo we will
 - Create an online git repository on gitlab
 - Clone the repository on our local machine
 - Add a Java Gradle project to the repository
 - Add and commit our files
 - Push the files to the online repository



Exercise 2



- Push exercise 1 to a gitlab repository
 - Make an account on gitlab (use your kdg email account!)
 - Create a project “persons” (you can first create a subgroup for it?)
 - Clone the project on your local machine
 - Copy your local project from exercise 1 into this empty cloned folder
 - Add all files
 - Commit the files
 - Push the files to gitlab
- Make a change to the project
 - Make some changes to the project
 - Add the change to the local git repository
 - Commit the changes to the local git repository
 - Push the changes to the online repository
 - Review the changes on gitlab



Agenda this week

Introduction

Where are we heading to...?

Java project with Gradle

Git

Lambdas and streams

Lambdas

Things labeled with @Extra are interesting, but we will not question it on the exam

@Extra

- Introduced in JDK 8 (2014)
- Introduces functional programming in Java
- **Makes it easy to pass a function as a parameter to another function (functions are *first-class citizens*)**
- Such functions are called *Lambda expressions*
- Lambdas make use of the arrow syntax: ->
- Advantages of lambdas
 - More readable code (for example event handling code)
 - Processing of collections is more intuitive (no more for-loops...)
 - Parallel processing is easier

Example of Lambda

```
button.setOnAction(event -> System.out.println("You clicked"));
```

- You used lambdas in JavaFX event handling!
- Before lambdas this was:

```
button.setOnAction(new ClickMeEventHandler());  
//...  
public class ClickMeEventHandler implements EventHandler<ActionEvent> {  
    @Override  
    public void handle(ActionEvent event) {  
        System.out.println("You clicked");  
    }  
};
```

General syntax

- Replace:

~~new SomeClass()~~

```
SomeClass implements SomeInterface {  
    @Override  
    public SomeType someMethod(args) {  
        body  
    }  
}
```

- With

(args) -> { body }

If you have single parameter, you can drop the (...) around args.
If you have no parameters, use empty brackets ()

Other example: Comparator interface

```
Arrays.sort(testStrings, new MyComparator());  
class MyComparator implements Comparator <String>() {  
    @Override  
    public int compare(String s1, String s2) {  
        return s1.length() - s2.length();  
    }  
});
```

Non-anonymous
implementation

```
Arrays.sort(testStrings, new Comparator<String>() {  
    @Override  
    public int compare(String s1, String s2) {  
        return s1.length() - s2.length();  
    }  
});
```

Anonymous implementation

```
Arrays.sort(testStrings,  
    (String s1, String s2) -> {  
        return s1.length() - s2.length();  
    }  
);
```

Lambda syntax

```
Arrays.sort(testStrings, (s1, s2) ->  
    s1.length()-s2.length());
```

If the implementation is a single return
statement you can omit the ~~return~~ ... }

Lambdas: remarks

- Lambdas can only be used with *functional interfaces*: interfaces **with only one method!**
 - Comparator and EventHandler are functional interfaces
- Lambdas are powerful when used in combination with *Streams* (see further)

Comparator.comparing

You're often comparing an object attribute. There is a helper function for this:

```
Arrays.sort(testStrings, Comparator.comparingInt (s -> s.length()));
```

.comparing calls a Comparable's compareTo() (getName() returns a String)

```
Arrays.sort(employees, Comparator.comparing(e -> e.getName()));
```

What if you're comparing on 2 attributes?

```
Arrays.sort(employees, Comparator  
    .comparingDouble ((Employee e) -> e.getLength())  
    .thenComparing(em -> em.getName()));
```

Comparator.comparing (continued)

Or sorting descending?

```
Arrays.sort(employees, Comparator.comparing((Employee e) -> e.getName())  
        .reversed());
```

demo_lambda

- In this demo we will
 - Show how to implement the Comparator interface with an inner class
 - Replace that implementation with a lambda



Lambdas: remarks

- You can create your own functional interfaces: add the `@FunctionalInterface` annotation.
- A functional interface can only have 1 method
- You seldom create your own functional interfaces: you can find a list of functional interfaces in `java.util.function`

```
@FunctionalInterface
public interface MyFunctionalInterface {
    double myMethod(int myParam);
}
```

Functional Interfaces

- In `java.util.function` you can find a list of functional interfaces
= interfaces with 1 method.
- Examples are:
 - `Function<T,R>` → represents a function with one argument of type T that returns a value of type R
 - `Consumer<T>` → represents a function with one argument of type T that does not return anything
 - `Supplier<T>` → represents a function with no arguments that returns a value of type T
 - `Predicate<T>` → represents a function with one argument of type T that returns a boolean
 - ...

Example: using a Function<String, Integer>

- We can create a lambda that returns the number of vowels in a String:

```
// Lambda expression to count the number of vowels in a string
Function<String, Integer> countVowels =
    str -> {
        int count = 0;
        for (char c : "aeiou".toCharArray()) {
            if (str.indexOf(c) != -1) count++;
        }
        return count;
    };
// Test the lambda expression String
testString = "Hello World";
int vowelCount = countVowels.apply(testString);
System.out.println(vowelCount);
```



Exercise 3: Lambda's

1. Create a functional interface with a method that takes an int as a parameter and returns a double
2. Write a function `becomes`, that receives the operation and the parameter, calls the operation and prints *parameter becomes result*
3. Call the function to calculate half of the number. Expected output for `becomes(i -> i/2, 6)`
6 becomes 3.00
4. Call `becomes` to calculate the surface of a circle with a given radius (surface = $\text{PI} \times \text{radius}^2$). Example output:
3 becomes 28.27



Exercise 3: Lambda's

5. Now write a function `tenFold` that takes the lambda as a parameter. It prints out `Calculating...` and then prints the result of applying the lambda to the numbers 0 to 10.
 6. Call `tenFold` with a lambda that calculates `ten` to the power of `n`. =>
 7. Call `tenFold` with a lambda that calculates the factorial of `n`.
- Expected result:

```
Calculating...
```

```
1.0 // 0!
```

```
1.0
```

```
2.0
```

```
6.0
```

```
24.0
```

```
120.0
```

```
720.0
```

```
5040.0
```

```
40320.0
```

```
362880.0
```

```
3628800.0
```

```
Calculating...
```

```
1.0 // 100
```

```
10.0
```

```
100.0
```

```
1000.0
```

```
10000.0
```

```
100000.0
```

```
1000000.0
```

```
1.0E7
```

```
1.0E8
```

```
1.0E9
```

```
1.0E10
```

Exercise 3: Lambda's



8. Make a list of 6 lambda's. The first calculates powers of 0 , the second powers of 1 and so on.
9. Run through the list and pass each lambda to `TenFold`. Partial expected result:

```
Calculating...
```

```
1.0
```

```
0.0
```

```
0.0
```

```
_ // 0n
```

```
Calculating_
```

```
1.0
```

```
1.0
```

```
_// 1n
```

```
Calculating...
```

```
1.0
```

```
2.0
```

```
4.0
```

```
8.0
```

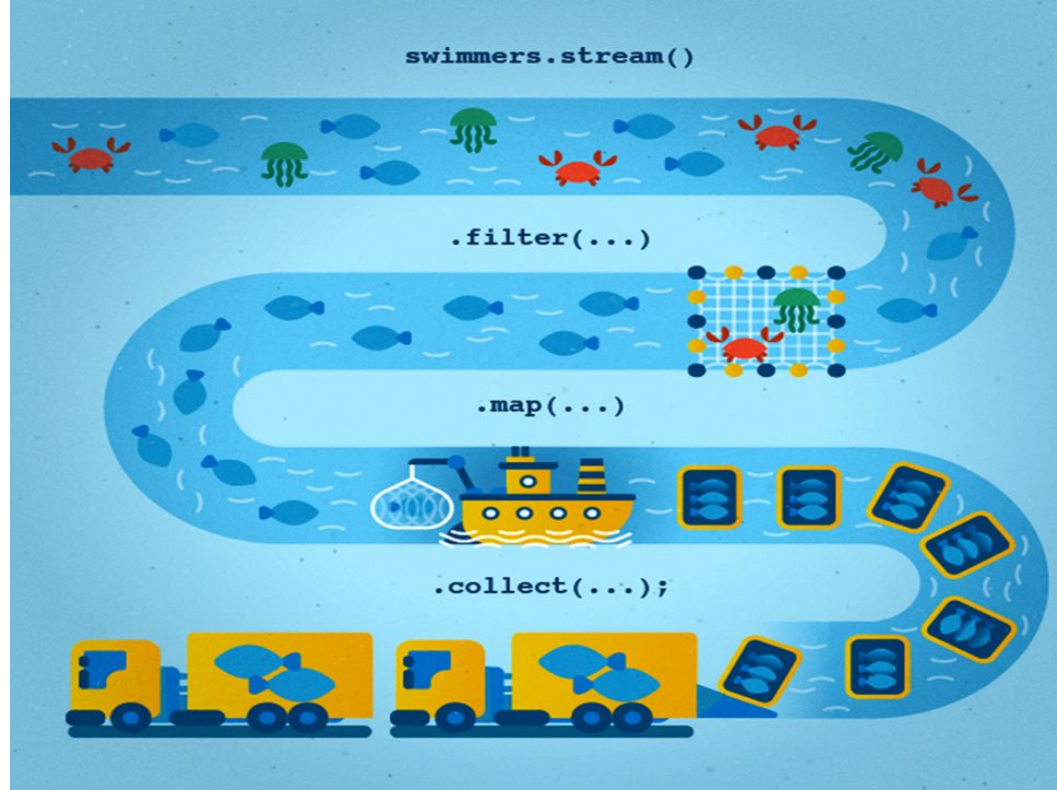
```
_ // 2n
```

```
Calculating...
```

```
_
```

Streams

- Collections can be turned into a *stream*
- You can define *operations* on the elements of a stream using lambdas
- Type of operations:
 - **Filtering** the stream of elements
 - **Transforming** the stream of elements
 - **Collecting** the stream of elements



Good streams tutorial: <https://stackify.com/streams-guide-java-8/>

Working with streams

1. Create a **stream**
2. Specify 0 or more **“intermediate operations”**
 - Operation on stream (filter, transform) → use lambda's for this!
 - Result is new stream
3. Specify a **“terminal operation”**
 - Consumes the stream: turns it into a final object



Most common used Functional Interfaces in streams:

Functional Interface	Single method	Typical use in streams:
<code>Predicate<T></code>	<code>boolean test (T t)</code>	Filtering the stream
<code>Function<T,R></code>	<code>R apply (T t)</code>	Mapping the stream
<code>Consumer<T></code>	<code>void accept (T t)</code>	Terminating the stream
<code>Supplier<T></code>	<code>T get ()</code>	Creating a stream
...	...	...

All these functional interfaces make use of **generics**!
This will be a topic one of the future lessons...

Example

Count the number of articles that cost more than €400

```
List<Article> articles = Articles.getArticles();
```

Classic loop:

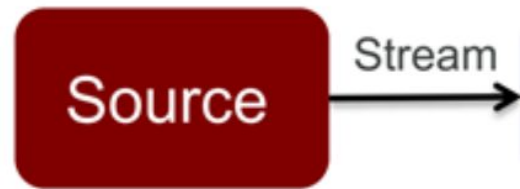
```
long count = 0;
for (Article article: articles) {
    if (article.getPrice() > 400.0) {
        count++;
    }
}
System.out.println(count);
```

With stream:

```
long count =
    articles.stream()
        .filter(a -> a.getPrice() > 400.0)
        .count();
System.out.println(count);
```

This lambda uses the **Predicate** functional interface

Examples of creation of streams



```
List<T> list = ...;  
list.stream()  
    .intermediate  
    .intermediate  
    ...  
    .terminal
```

```
T[] array = ... ;  
Stream.of(array)  
    .intermediate  
    .intermediate  
    ...  
    .terminal
```

This lambda uses the **Supplier** functional interface

```
Stream<Integer> numberStream = Stream.generate(()->random.nextInt(100,901))
```

```
Stream<Integer> numberStream = Stream.iterate(1, n -> n * 2)
```

Intermediate operations



- `map`: transform the data. Uses a Function
- `filter`: some elements pass through. Uses a Predicate.
- `sorted`: sorts the stream. Uses a Comparator.
- `limit`: limits to a certain number. No lambda used.
- ...

```
Stream.of(1, 2, 3, 4, 5)
    .map(i -> i * i)
    .forEach(e -> System.out.print(e + " "));
```

This lambda uses the **Function** functional interface

This lambda uses the **Consumer** functional interface

```
Stream.of(1,2,3)
    .filter(n -> n % 2 == 1)
    .filter(n -> n < 3)
    .forEach(a -> System.out.print(a + " "));
```

Terminal operations



- `forEach`: runs the lambda on all elements. Use a Consumer.
- `findFirst`, `findAny`: returns Optional of first/any element.
- `collect`: collect into a List, Set, Map, String, ...
 - Uses `Collectors.toList()`, `Collectors.toSet()`, ... as parameter.
- `toList`: **same as** `collect(Collectors.toList())`;
- `reduce`: Combines elements in single result.
- Specialised aggregators: `min`, `max`, `average`, `sum`, `count`
- ...

Terminal operations



- Do not return a stream.
- Example: reduce is a versatile aggregate operation that returns one result

```
System.out.println("Cheapest you can get: " + articles.stream()  
    .reduce( articles.get(0),  
        (cheap,article) -> cheap.getPrice() < article.getPrice()? cheap: article)  
    );
```

Initial value

intermediate
result

returns new
intermediate result
for next iteration

- Often you can get the same result using the simpler specialised aggregators.
Exercise: find the cheapest article using min!

Optional

- Wrapper for Nullable Values: Optional is a container object that may or may not contain a non-null value. It helps to avoid NullPointerException by handling the presence or absence of values more expressively
- Methods of Optional:
 - `isPresent(), isEmpty()`: Checks if the Optional contains a value.
 - `ifPresent(Consumer<T> action)`: Executes the provided action if a value is present.
 - `ifPresentOrElse(Consumer<T> action, Runnable alternate)`: executes action if present or else alternate
 - `get()`: Retrieves the value. Always check if it is present first! Throws NoSuchElementException if empty.
 - `orElse(T other)`: Returns the value if present; otherwise, returns the specified default value.
 - `orElseGet(Supplier<T> action)`: Returns the value if present; otherwise, returns the result of the provided action
 - ...
- Has static methods to create your own Optionals
 - `of(T value), ofNullable(T value), empty()`Option
- [A guide to Optional](#)

Optional: an example

- The `findFirst` method of a Stream will return an object of type `Optional`.
- This way we avoid running into `NullPointerExceptions` in streams

```
Optional<String> firstElement = Stream.of("apple", "banana", "walnut")  
    .filter(fruit-> fruit.contains("n"))  
    .findAny();  
firstElement.ifPresent(System.out::println); // probably prints banana  
//if we do not find a fruit with a "n" this won't print anything  
//(and will not throw a NullPointerException!)
```

This lambda uses the Method Reference syntax.

Method reference examples

```
articles.stream()  
    .filter(a -> a.getPrice() > 400.0)  
    .forEach(a -> System.out.println(a))
```



```
articles.stream()  
    .filter(a -> a.getPrice() > 400.0)  
    .forEach(System.out::println)
```

```
Stream.of("Dirk", "Annelies", "Thomas")  
    .sorted((s1,s2)->s1.compareTo(s2))  
    .forEach(System.out::println);
```



```
Stream.of("Dirk", "Annelies", "Thomas")  
    .sorted(String::compareTo)  
    .forEach(System.out::println);
```

An instance method reference is called ON the first parameter of the lambda, subsequent lambda parameters are passed as arguments

reduce revisited

reduce without an initial value,
uses the first element in the array
as initial value and starts
iterating on the second

```
articles.stream()  
  .reduce( (cheap,article) -> cheap.getPrice() < article.getPrice()?cheap:article)  
  .ifPresentOrElse(  
    article -> System.out.println("Sale! Buy " + article),  
    () -> System.out.println("No cheapskates"));
```

this version returns an Optional

this is executed if no article is found



Streams: remarks

- Streams are *lazy*: intermediate operations are not evaluated until the terminal operation is invoked
- Better alternative for for-loops. We will try to avoid for-loops in combination with collections.
- Automatic parallelization possible: you can make a stream parallel and it will try to perform the operations in parallel!
- Stream are not IO Streams...
- If the lambda is a single method invocation, in some cases you can use a *method reference*.

demo_streams

- In this demo we will
 - Create a stream of Articles
 - Demonstrate intermediate operations like filter, map, sorted
 - Demonstrate terminal operations like forEach, sum, collect, ..

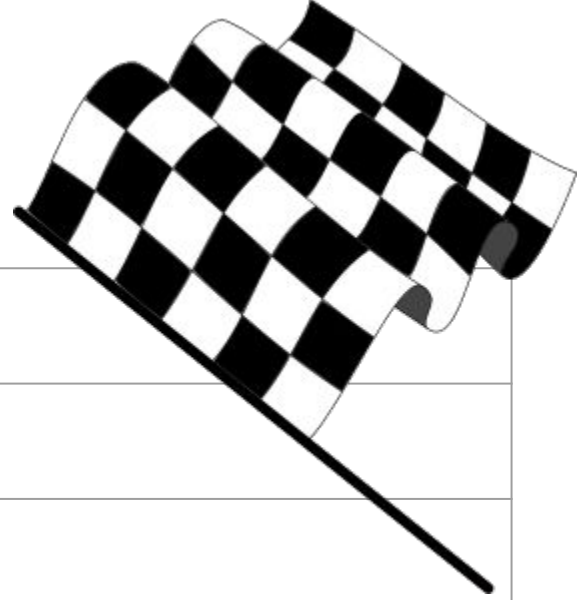


Exercise 4



- Create a class Actor. An actor has a name, gender (m,f) and a birth year.
- Create a static factory method createRandomActor(), that creates a random Actor (names are “keanu1,2,3...” or “reece1,2,3...”, birth year between 1920 and 2010)
- Use streams to (NO classic loops!)
 - Print out a list of 20 random actors, save in a List<Actor>
 - Filter out all female actors and print
 - Filter out all male actors that are older than 50 and print
 - Make a sorted list, sorted by age and print
 - Give the total age of all female actors
 - Make a string with all the first letters of the male actors

Finished!



Introduction
Where are we heading to...?
Java project with Gradle - Git
Lambdas and streams